Session 7 · Deck 7.A3

Backpropagation, the Loss, the Optimizer, and Three Samples

Zhigang Feng

Thursday, November 5, 2026

© Zhigang Feng Redistribution prohibited without permission

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 7.A3 ▼

S1 ✓	S2 ✓	S3 ✓	S4 ✓	S5 ✓	S6 ✓	S7	S8	S9	S10
Sep 24	Oct 1	Oct 8	Oct 15	Oct 22	Oct 29	Nov 5	Nov 12	Nov 19	Dec 3
Intro	HA · OLG	Num. DP	Numerics	Numerics	HA classic	ML	RL	Text	Agents
Growth	Policy	Python	I	II	The wall	Deep nets	HA + DL	LLMs	Projects
trunk	branches	the machine · classical methods			the wall	new tools		new data	colleagues

Build it on paper → solve it on the machine → push it to the frontier with AI

Block A — learning a function: 7.A1 the learning problem and the neuron · 7.A2 activations and approximation · **7.A3** backpropagation and training. **Block B — using it:** 7.B1 PyTorch and a forecast · 7.B2 the growth model · 7.B3 architecture and shape · 7.B4 capacity and order.

© Zhiwei Fan Redistribution prohibited without permission

Roadmap: Four Parts

7.A2 showed that a network can represent the function. This deck finds the parameters, and says when to stop.

Part	Topic	The question it answers
1	The problem, and the learning loop	What is being minimised, and which four steps does every network repeat?
2	Backpropagation, with numbers	One observation, two layers, every derivative by hand: what does the recursion compute, and what does it cost?
3	The loss, and the optimizer	What does the criterion assume, and what do you do with a gradient that is noisy — with Newton out?
4	The dataset, and three samples	Where do the data come from, and what makes a reported number mean something?

© Zhigang Feng Reproduction prohibited without permission

Idea

The chain rule, run backwards, gives every derivative for about the price of one evaluation. A step against it is the whole optimizer. Three samples, each with one job, are the whole of honesty.

An empirical loss with no formula for the minimiser, and not convex.

- 7.A2 fixed a family of functions f_θ and showed that a good member exists. Training chooses θ:

$$\min_\theta \ \mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell\big(f_\theta(x_i), y_i\big).$$

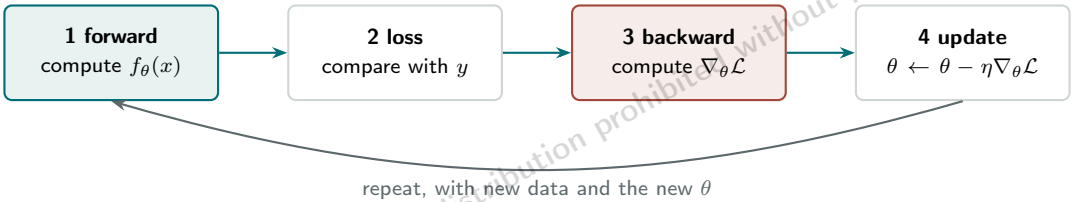
- The **loss** ℓ is a number that is small when the prediction $f_\theta(x_i)$ is close to the target y_i.
- θ has 10^3 to 10^9 entries, and $\mathcal{L}$ is not convex in θ. There is no formula for the minimiser, and which minimum training finds matters: 7.B4 gets to that.
- The only affordable information is the **gradient** $\nabla_\theta \mathcal{L}$: the direction in which the loss rises fastest.
- **Three questions organise this deck: how to get the gradient cheaply, what to do with it, and how to know the answer is worth anything.**

© Zhiyang Feng • Redistribution prohibited without permission

Miao & Feng, *AI for Economists*, §9.3.1.

The Learning Loop: Forward, Loss, Backward, Update

Every network in this course is trained by four steps, repeated.

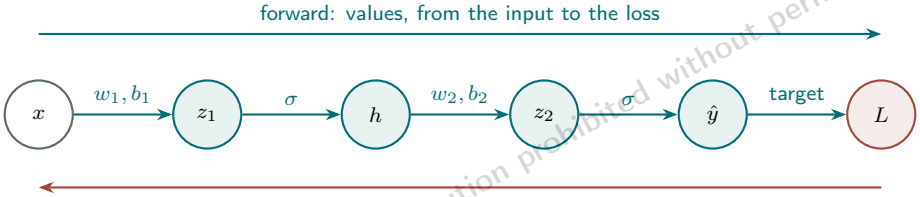


© Zhirang Feng • Redistribution prohibited without permission

- The **forward pass** evaluates the network, layer by layer, from input to prediction.
- The **backward pass** returns the derivative of the loss with respect to every parameter.
- The **learning rate** $\eta > 0$ is the step length: the one free choice in the update.
- **Every network in this course is trained by these four steps, repeated.**

1 · One Graph, Two Traversals

One graph, traversed twice. Backward reuses what forward stored.

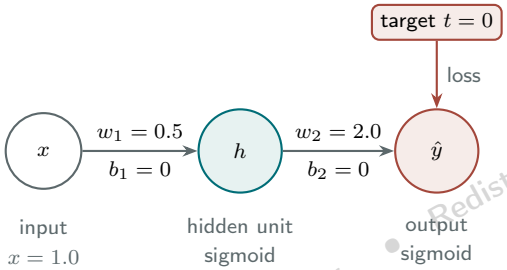


Redistribution prohibited without permission

- A **computation graph** records each intermediate value (node) and the operation behind it (edge): 3.B3's autograd tape.
- Backward walks the same edges in reverse, applying the **chain rule** edge by edge.
- The colour convention of this deck: **teal** and grey are values moving forward, **red** is derivatives moving back.
- **One graph, traversed twice. Backward reuses what forward stored.**

© Zhigang Feng

One input, one hidden unit, one output: every derivative can be computed by hand.

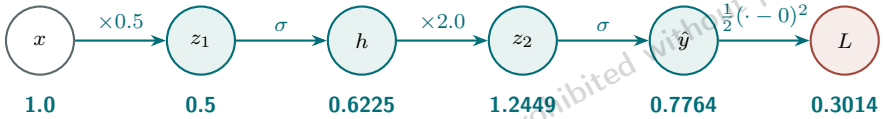


- The smallest honest version of 7.A1's digit classifier: one input, one hidden unit, one output.
- $\hat{y} \in (0, 1)$ is the predicted probability that the image is a "3". The target is $t = 0$: it is not.
- Loss: $L = \frac{1}{2}(\hat{y} - t)^2$, squared error, so that every step can be seen.
- **The arithmetic for one observation is the arithmetic for millions: forward, loss, backward, update.**

© Zhigang Feng
Redistribution prohibited without permission

Forward, with Numbers

Input $x = 1.0$, one sigmoid hidden unit, one sigmoid output unit, target $t = 0$.

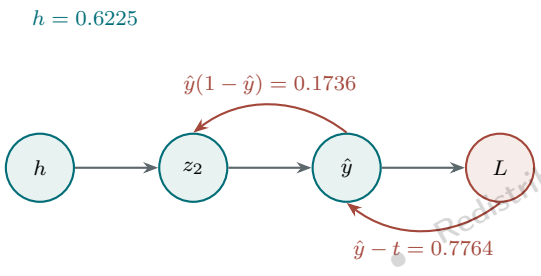


- $z_1 = w_1 x + b_1 = 0.5$, and $h = \sigma(0.5) = 0.6225$: the hidden unit is "62% on".
- $z_2 = w_2 h + b_2 = 1.2449$, and $\hat{y} = \sigma(1.2449) = 0.7764$.
- $L = \frac{1}{2}(0.7764 - 0)^2 = 0.3014$. The network says "78% a 3"; the truth is that it is not.
- **Learning means moving w_1 and w_2 so that $\hat{y}$ falls. By how much each? That is the backward pass.**

© Zhiyang Fang · Redistribution prohibited without permission

Lab 7, block A2; every number is asserted there against PyTorch's automatic differentiation.

Chain the links $w_2 \to z_2 \to \hat{y} \to L$, and name the part that will be reused.

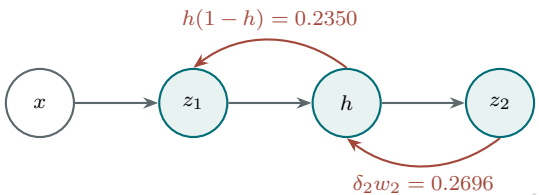


- Chain the links from w_2 to L:
  $\frac{\partial L}{\partial w_2} = \underbrace{(\hat{y} - t)}_{0.7764}\ \underbrace{\hat{y}(1-\hat{y})}_{0.1736}\ \underbrace{h}_{0.6225}$.
- Name the first two factors:
  $\delta_2 = \partial L/\partial z_2 = 0.1348$. It will be reused.
- Then $\partial L/\partial w_2 = \delta_2 h = 0.0839$ and $\partial L/\partial b_2 = \delta_2 = 0.1348$.
- **The sign is positive: raising w_2 raises the loss, so descent lowers w_2.**

© Zhigang Feng Redistribution prohibited without permission

Lab 7, block A2.

Carry δ_2 back through w_2 and the hidden activation; then step.



$\delta_1 = \partial L/\partial z_1 = \mathbf{0.0633}$

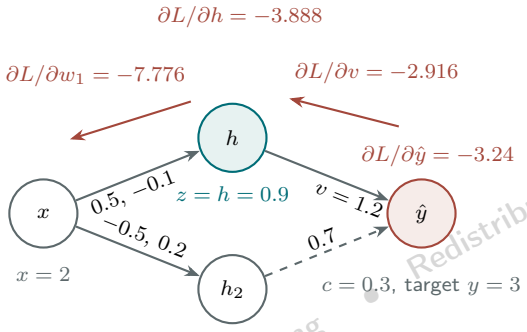
- Carry δ_2 back through w_2: $\partial L/\partial h = \delta_2 w_2 = 0.2696$. Then through the activation: $\delta_1 = 0.2696 \times 0.2350 = 0.0633$.
- $\partial L/\partial w_1 = \delta_1 x = 0.0633$.
- Update with $\eta = 0.1$: $w_2 \leftarrow 2.0 - 0.1(0.0839) = 1.9916$ and $w_1 \leftarrow 0.5 - 0.1(0.0633) = 0.4937$.
- **Both sigmoid slopes are below 0.25. Each extra sigmoid layer shrinks the signal again: 7.A2's vanishing gradient.**

© Zhigang Feng
Redistribution prohibited without permission

Lab 7, block A2.

The Same Thing with ReLU — and a Unit That Gets No Gradient

The recipe is identical; only $\partial h/\partial z$ changes, to 1 or 0.

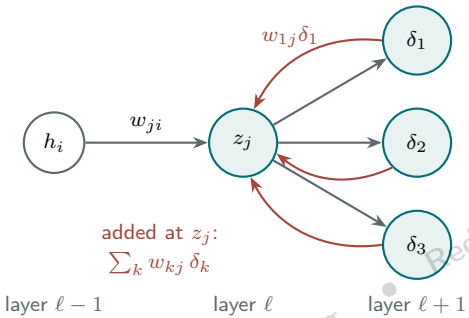


© Zhiyang Feng
Redistribution prohibited without permission

- ReLU unit, $L = (\hat{y} - y)^2$. Forward: $\hat{y} = 1.2(0.9) + 0.3 = 1.38$, so $L = 2.6244$.
- Backward, in red: each derivative is the previous one times a local slope. ReLU's slope is 1: no shrinkage.
- $\eta = 0.01$: $w_1 \to 0.5778$, $v \to 1.2292$; the loss falls to 1.7488 (7.B1 runs it in PyTorch).
- **The second unit has $z < 0$: slope 0, gradient exactly 0. If that holds for every observation it never learns: a "dying ReLU".**

Miao & Feng, *AI for Economists*, §9.3.2; Lab 7, block A2.

Add the messages from above, multiply by the local slope.



- The **error signal** of a unit is $\delta_j = \partial L/\partial z_j$. At the output, $\delta = \hat{y} - y$ for squared error.
- **Back one layer:** add the messages from above, multiply by the local slope: $\delta_j = \sigma'(z_j)\sum_k w_{kj}\,\delta_k$.
- **A gradient:** $\partial L/\partial w_{ji} = \delta_j\, h_i$, signal at the weight's output end times activation at its input end.
- **Backpropagation is the chain rule with each intermediate derivative computed once and reused.**

© Zhigang Feng
Redistribution prohibited without permission

D. Rumelhart, G. Hinton, R. Williams, "Learning representations by back-propagating errors", *Nature* 323, 1986.

The Same Recursion, in Matrices

Transpose the same weights, multiply by the local slope: two lines do every layer.

- Forward, for layers $\ell = 1, \ldots, L$: $z^{(\ell)} = W_\ell h^{(\ell-1)} + b_\ell$ and $h^{(\ell)} = \sigma\big(z^{(\ell)}\big)$, with $h^{(0)} = x$.
- Backward, from the top down ($\odot$ multiplies entry by entry):

$$\delta^{(\ell)} = \big(W_{\ell+1}^\top \delta^{(\ell+1)}\big) \odot \sigma'\big(z^{(\ell)}\big), \qquad \frac{\partial L}{\partial W_\ell} = \delta^{(\ell)} \big(h^{(\ell-1)}\big)^\top, \qquad \frac{\partial L}{\partial b_\ell} = \delta^{(\ell)}.$$

- The backward pass uses the *transpose of the same matrix* that was used going forward.
- σ' multiplies at every layer, so the activation decides whether the signal survives depth.
- $h^{(\ell-1)}$ is needed on the way back, so forward activations must be stored: the memory cost of training.
- **Two lines of linear algebra per layer give every derivative in the network.**

© Zhiqiang Feng · Redistribution prohibited without permission

Miao & Feng, *AI for Economists*, §9.3, eqs. (9.20)–(9.21).

Backpropagation Is the Adjoint Method

δ is a shadow price, and the backward pass is the costate equation.

- $\delta^{(\ell)}$ answers: by how much would the loss change if $z^{(\ell)}$ were perturbed? That is a **shadow price**.
- Treat each layer's equation $z^{(\ell)} = W_\ell h^{(\ell-1)} + b_\ell$ as a constraint. $\delta^{(\ell)}$ is its Lagrange multiplier: 1.B1's Lagrangian and 5.A1's implicit-function argument, met a third time.
- The backward recursion is a costate equation, solved from a terminal condition backwards, as in optimal control. There it is called the **adjoint method**; in numerical analysis, **reverse-mode differentiation**.
- A diagnosis follows: $\delta \approx 0$ in a layer means nothing below it can be improved at the current parameters. Printing gradient norms by layer is standard practice.
- **You already know how to read a multiplier. Backpropagation computes all of them in one backward sweep.**

© Zhigang Feng, 2026. Redistribution prohibited without permission

Miao & Feng, *AI for Economists*, §9.3.2; A. Griewank and A. Walther, *Evaluating Derivatives*, SIAM, 2008.

What a Gradient Costs

About $6P$ operations per observation per pass: FLOPs $\approx 6PnE$.

- Forward: about $2P$ operations per observation — a multiply and an add per weight. Backward, about $4P$: it returns two gradients per layer, for the weights and for the layer's input.
- **All P derivatives at once cost a small multiple of *one* forward pass, whatever P is.** A numerical Jacobian needs $P+1$ passes; a Hessian has P^2 numbers, impossible at $P = 10^6$.
- Budget $6PnE$ for n observations and E epochs: $P = 10^5$, $n = 5 \times 10^4$, $E = 100$ give 3×10^{12} operations. Stored activations (batch $\times$ width $\times$ depth) cap the batch size, not statistics.
- A chip's advertised peak is never reached: measured utilisation is 21.3% for GPT-3 and 46.2% for PaLM-540B. Multiply a runtime worked out from peak by two to four.
- **A macroeconomic network on a state vector is a small computation.**

© Zhigang Feng · Redistribution prohibited without permission

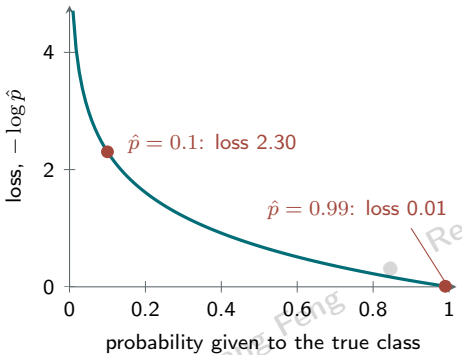
Miao & Feng, ***AI for Economists***, §9.3.4; A. Griewank and A. Walther, ***Evaluating Derivatives***, SIAM, 2008; J. Kaplan et al., arXiv:2001.08361, 2020 (the $6P$ convention); A. Chowdhery et al., ***JMLR*** 24, 2023 (utilisation).

A loss is an assumption about the noise, not a convention.

- For a continuous target, the mean squared error: $\mathcal{L} = \frac{1}{n}\sum_i (\hat{y}_i - y_i)^2$.
- Minimising it is maximum likelihood under Gaussian errors. For a linear f_θ it *is* ordinary least squares.
- A loss is therefore an assumption about the noise, not a convention. Change the noise model and the loss changes: Poisson likelihood for counts, the check function for quantiles.
- In 7.B1 the target is next month's yield. In 7.B2 the "prediction" is an Euler-equation residual and the target is zero, so no data enter the loss at all.
- **Squared error asks for the conditional mean. If that is not what you want, it is the wrong loss.**

© Ehigiator Fevre. Redistribution prohibited without permission

Scores become probabilities; the loss is minus the log-probability of the true class.

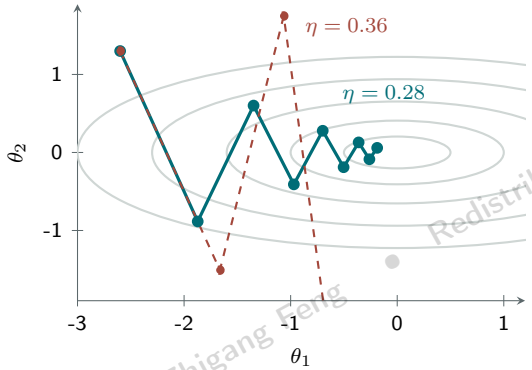


- A classifier returns a score z_k for each of K classes. The **softmax** turns scores into probabilities: $\hat{p}_k = e^{z_k} / \sum_j e^{z_j}$.
- **Cross-entropy** is the negative log-probability given to the true class, $\mathcal{L} = -\log \hat{p}_{k^\star}$.

© Zhigang Feng · Redistribution prohibited without permission

D. McFadden, "Conditional logit analysis of qualitative choice behavior", in *Frontiers in Econometrics*, 1974.

The learning rate is limited by the steepest direction, and progress by the flattest.



© Zhigang Feng
Redistribution prohibited without permission

- **Gradient descent**: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$. Step against the gradient.
- The picture is $\mathcal{L} = \frac{1}{2}(\theta_1^2 + 6\theta_2^2)$. The teal path zigzags across the steep direction and converges.
- The red path uses a step only slightly longer. It overshoots further each time and diverges.
- **The learning rate is limited by the steepest direction, and progress by the flattest.**

3 · Why Not Newton — 4.A2 Already Answered This

N parameters, N^2 Hessian entries: at $N = 10^6$, a trillion numbers.

N	Newton	L-BFGS	CG
10^2	10^4	2,000	300
10^4	10^8	2×10^5	3×10^4
10^6	$\mathbf{10^{12}}$	2×10^7	3×10^6

Numbers stored: the Hessian; ten stored pairs of vectors; three vectors.

- A Newton step needs the Hessian: N^2 entries for N parameters, stored and factored.
- $N = 10^6$ is the parameter count of a *small* network: 10^{12} numbers.
- 4.A2 drew the conclusion: nobody trains a network with Newton's method, and Session 7's optimizer is first-order.
- L-BFGS with ten stored pairs needs 2×10^7 numbers at $N = 10^6$ — affordable, and still not used, for the reason on the next frame.
- **From here on the optimizer uses gradients only.**

© Zhigang Feng
Redistribution prohibited without permission

J. Nocedal and S. Wright, *Numerical Optimization*, 2nd ed., 2006, ch. 7. The table is 4.A2's.

3 · And Why Not Conjugate Gradient Either

CG and L-BFGS need the same gradient every time they ask; a mini-batch is a new function every step.

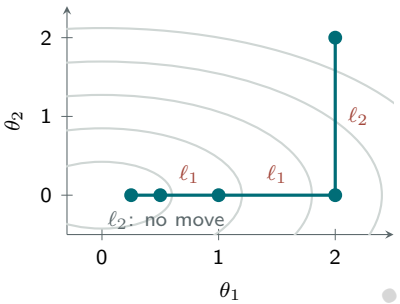
- 4.A2's other result: conjugate gradient is exact in at most N steps on an N-dimensional quadratic, storing a handful of vectors. On the two-dimensional example it reached the optimum from $(2, 2)$ in two steps, through $(0.889, -0.222)$.
- CG and L-BFGS both need a gradient that is the same function every time they ask: line searches and conjugacy corrections assume a fixed objective.
- A mini-batch gradient is a different function at every step. The conjugacy CG builds is destroyed by the noise before it can be used.
- Second-order and conjugate methods buy accuracy per step and pay in memory and in the need for an exact gradient. Training wants the opposite: very many cheap, noisy steps.
- The exception proves it: full-batch L-BFGS is a sensible choice for a small collocation problem.
- **The noise is not merely tolerated: the next two frames argue that it helps.**

© Zhiqiang Feng · Redistribution prohibited without permission

J. Nocedal and S. Wright, *Numerical Optimization*, 2nd ed., 2006, chs. 5 and 7. The two-step example is 4.A2's.

Why a Sampled Gradient Still Gets There

Differentiate a random subset of the loss's terms: unbiased, noisy, and cheap.



- **Stochastic gradient descent** (SGD): differentiate a random subset of the loss's terms at each step.
- Here $\ell_1 = \frac{1}{2}\theta_1^2$, $\ell_2 = \theta_2^2$, step 0.5: only the drawn coordinate moves — 4.A2's example, the same five steps.
- Unbiased: its expectation is proportional to the full gradient; its noise grows like $\sqrt{N}$ in the parameter count (4.A2).
- With $\sum_n \eta_n = \infty$, $\sum_n \eta_n^2 < \infty$ (Robbins–Monro, 1951) SGD settles at a stationary point — not a global minimum.
- **The path is not monotone and is not meant to be — and the goal is not the lowest training loss but a minimum that generalises.**

© Zhigang Feng
Redistribution prohibited without permission

L. Bottou, F. Curtis, J. Nocedal, *SIAM Rev.* 60(2), 2018; A. Choromanska et al., AISTATS 2015; T. Garipov et al., NeurIPS 2018; H. Robbins and S. Monro, *Ann. Math. Statist.* 22(3), 1951.

How many terms of the sum do you look at before you move?

Method	Terms per step	Cost of a step	Path
full batch	all n	high	smooth
stochastic	1	lowest	very noisy
mini-batch	32 to 1,024	one matrix product	noisy, averaged

- An **epoch** is one pass through the training data; an **iteration** is one update. With 40,000 observations and batches of 256, an epoch is 156 full batches.
- Why mini-batch: many updates per pass, activations that fit in memory, hardware built for matrix products.
- The gradient noise also helps the iterate leave saddle points and shallow minima.
- In 7.B2 there is no dataset, so a "batch" is a fresh draw of 256 states at every step.

© Zhigang Feng · Redistribution prohibited without permission

Adaptive Step Sizes: RProp → RMSProp → Adam

Adam is stochastic gradient descent with a per-parameter step size — the optimizer of 7.A2, 7.B1 and 7.B2.

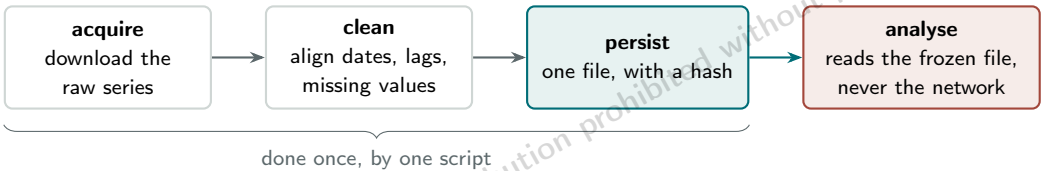
- **Momentum**: step along an exponentially weighted average of past gradients. An economist would call it adaptive expectations about the descent direction.
- **RProp** (1993) sized each parameter's step from the sign of its gradient. **RMSProp** (2012) made that work with mini-batches: divide each step by a running average of that parameter's squared gradients.
- **Adam** (2015) is momentum plus RMSProp, with decay rates 0.9 and 0.999 by default.
- **AdamW** (2019) applies **weight decay**, a shrinkage of the weights at each step, outside the adaptive scaling. It is today's default. Later proposals: Lion, Sophia, schedule-free methods.
- **Adam is SGD with a step size for each parameter. It is still first-order: no Hessian is formed or stored.**

© Zheng Feng · Redistribution prohibited without permission

D. Kingma and J. Ba, "Adam", ICLR 2015; I. Loshchilov and F. Hutter, "Decoupled weight decay regularization", ICLR 2019; M. Riedmiller and H. Braun, IEEE ICNN 1993.

The Data Workflow: Acquire, Clean, Persist — Once

A result is reproducible only if its data are frozen.

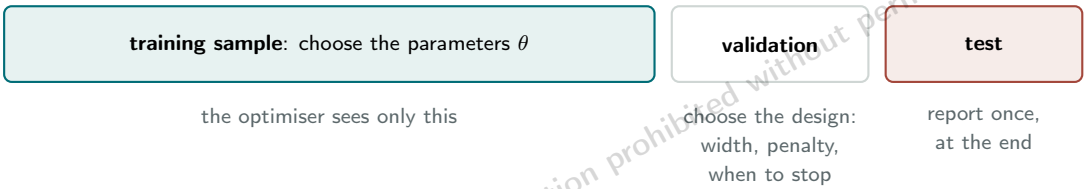


© Zhirang Feng · Redistribution prohibited without permission

- Statistical agencies revise. A notebook that downloads on every run gives different answers on different days.
- Separate the download from the analysis; the analysis then runs offline and gives the same numbers every time. 3.B4's acquire–clean–analyse loop, with the freeze made explicit.
- **A result is reproducible only if its data are frozen.**

4 · Train, Validation, Test: Three Jobs, Three Samples

Each sample does one job. A sample used for two jobs does neither honestly.



© Zhiang Fang · Redistribution prohibited without permission

- A flexible model can fit its training sample as closely as you like. That says nothing about new data.
- **Validation** data choose between designs, so they are no longer a clean test.
- **Test** data are touched once. Their error estimates the error on future data.
- With time series the three samples are consecutive periods, never a shuffle.
- **Each sample does one job. A sample used for two jobs does neither honestly.**

Leakage: Four Ways the Test Set Gets into Training

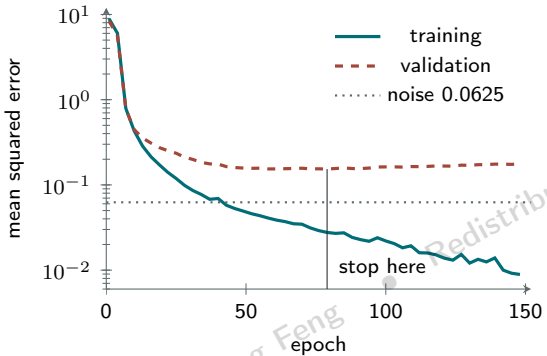
Ask of every reported accuracy: what did the model already know about these cases?

- **Leakage**: information from the test sample reaches the fitted model, so the test error is too good.
- **Shuffling a time series.** Neighbouring months are nearly identical; a shuffled split puts the answer for March next to the question for April.
- **Scaling with full-sample statistics.** The mean and standard deviation used to standardise the inputs must come from the training window alone.
- **Tuning on the test set.** Trying ten designs and reporting the best test error makes the test set a validation set.
- **The same template on both sides.** In Lab 7, sentences built as subject × phrase score 0.97 on a random split and 0.50 when whole phrases are held out.
- **Ask of every reported accuracy: what did the model already know about these cases?**

© Zhigang Feng · Redistribution prohibited without permission

Accuracies: Lab 7, appendix (logistic regression on word counts, 96 sentences).

A training loss below the noise floor is a diagnosis, not an achievement.

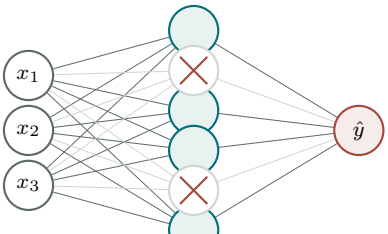


- **Underfitting**: both curves high. **Overfitting**: training loss falls while validation loss turns up.
- $n = 2{,}000$. Training loss ends at 0.0089, below the noise variance 0.0625: memorised noise.
- **Early stopping**: keep the best-validation parameters, near epoch 80.
- **A training loss below the noise floor is a diagnosis, not an achievement.**

© Zhigang Feng Redistribution prohibited without permission

Recorded in this course's earlier computations (2026); not recomputed in Lab 7.

A unit cannot count on any partner being present, so each must carry information on its own.



$p = \frac{1}{3}$: two of six units off; a new draw at every step

- **Dropout**: each step, each hidden unit is kept with probability $1 - p$, else zeroed: $\tilde{h} = m \odot h/(1-p)$, $m_j \sim \text{Bernoulli}(1-p)$, redrawn each step.
- The $1/(1-p)$ keeps $E[\tilde{h}] = h$; at prediction nothing is dropped and the full network runs once (`eval()` in 7.B1).
- $p = 0.1$ to 0.2 on hidden layers, never on the output.
- **A unit cannot count on any partner being present, so each must carry information on its own.**

© Zhigang Feng
Redistribution prohibited without permission

N. Srivastava et al., "Dropout: a simple way to prevent neural networks from overfitting", *JMLR* 15, 2014.

Why Dropout Regularises

An ensemble for the price of one model, and a ridge penalty in disguise.

- With m hidden units there are 2^m thinned sub-networks; each step trains one of them, and all share one set of weights.
- At test time the full network approximates the average of those sub-networks: an ensemble for the price of one model.
- For a linear model, dropout on the inputs is a ridge penalty scaled by each regressor's second moment: an adaptive weight decay (Wager, Wang and Liang, 2013).
- It is noise on purpose: the network is trained to survive losing any one feature, as new data will make it.
- Cost: a noisier gradient and more steps; at $n = 50{,}000$ the next frame shows it is worth 1%.
- **Dropout pays where the data are too few to pin the weights down, and that is the economist's usual case.**

© Zhiqiang Feng · Redistribution prohibited without permission

S. Wager, S. Wang, P. Liang, "Dropout training as adaptive regularization", *NeurIPS* 26, 2013; Srivastava et al. (2014).

Three Instruments Against Overfitting, and What They Are Worth

Dropout pays where data are scarce. More data does more.

Strategy	best validation $n = 2{,}000$	best validation $n = 50{,}000$
none	0.1535	0.0788
weight decay 10^{-4}	0.1536	0.0804
dropout 0.1	0.1257	0.0780
dropout 0.2	0.1341	0.0792
both (0.1)	**0.1238**	0.0780

Noise floor 0.0625; 30 inputs, 17 pure noise.

- **Weight decay**: a penalty $\lambda_w \|\theta\|^2$, which is ridge regression.
- **Dropout**, from the previous two frames: a random share p of hidden units off at each step.
- Early stopping acts like a ridge penalty of about $1/(\eta T)$ after T steps.
- **Dropout pays where data are scarce: 18% at $n = 2{,}000$, 1% at $n = 50{,}000$. More data does more: 0.1535 to 0.0788.**

Recorded in this course's earlier computations (2026), median of three seeds; not recomputed in Lab 7.

© Zhigang Feng · Redistribution prohibited without permission

Takeaways

- **Part 1.** Training minimises an empirical loss with no formula for the minimiser and no convexity. Every network is trained by four steps, repeated — forward, loss, backward, update — on one graph, traversed twice.
- **Part 2.** Backpropagation is the chain rule with each intermediate derivative computed once and reused: $\delta^{(\ell)} = (W_{\ell+1}^{\top}\delta^{(\ell+1)}) \odot \sigma'(z^{(\ell)})$, a shadow price per layer. Two networks small enough to check agree with `autograd`, and all P derivatives cost a small multiple of one forward pass: about $6PnE$ operations in all.
- **Part 3.** A loss is an assumption about the noise: squared error is OLS under Gaussian errors; cross-entropy is the log-probability of the true class. The optimizer steps against the gradient, first-order because of 4.A2's memory wall; a sampled gradient is unbiased, so mini-batches still get there, and Adam sizes each parameter's step.
- **Part 4.** Each sample does one job: the optimiser sees the training data, the design is chosen on validation data, and the test data are touched once. Generalisation is measured, not assumed: a training loss below the noise floor is the mark of overfitting, and regularisation pays mainly when data are scarce. 7.B4 returns with a case the noise floor cannot diagnose at all.

© Mitesh Patel · Distribution prohibited without permission

Bridge: The Same Loop, in Code

Block A built the object (7.A1–7.A2) and the procedure (this deck) on paper and on one picture. Everything here is four library calls.

Block B makes them: first where the answer is known, then on a real economic series, then on a model with no data at all.

Next: 7.B1

The same loop in PyTorch: tensors, the autograd tape, `backward()` checked against this deck's ReLU example, a curve whose law we wrote ourselves, and next month's 10-year Treasury yield.

© Zhigang Feng
Redistribution prohibited without permission